

Lab 07 - Deployments, Rollout Strategy and Rollback

Lab id	Lab 07
Day / Topic	Day 2 - Workloads & Scheduling
Duration	55 minutes
Namespace	kcna-lab07
Mapping	LO2 *Identify the technical and practical requirements in a Kubernetes setup* A3 *Identify technical and practical requirements as well as stakeholders' demands* K3 *Objectives of solution architecture*
Cluster	Single-node kind cluster, Kubernetes v1.30+

1. Objective

By the end of this lab you will be able to:

1. Explain the three-level ownership chain Deployment → ReplicaSet → Pod and show it with ownerReferences.
2. Compute, in advance, the ceiling (replicas + maxSurge) and floor (replicas - maxUnavailable) of a rolling update, then observe the cluster obey both.
3. Use kubectl rollout status, history, undo and --to-revision, and annotate revisions with kubernetes.io/change-cause.
4. Diagnose a rollout wedged by an unpullable image, read ProgressDeadlineExceeded, and recover without downtime.
5. State why .spec.selector is immutable on an existing Deployment and recognise the rejection message.

2. Prerequisites

```
kubectl version --output=yaml | grep gitVersion | head -1
kubectl get nodes
```

Expected output:

```
gitVersion: v1.31.0
NAME                STATUS    ROLES    AGE   VERSION
kind-control-plane  Ready    control-plane  34m   v1.31.0
```

Lab 06 should be complete (labels and selectors are assumed knowledge). Work from the lab directory:

```
cd courseware/labs/lab-07-deployments-rollout-rollback
ls manifests data
```

Expected output:

```
data:
release-plan.csv■site-r24.1.html■site-r24.2.html

manifests:
00-namespace.yaml■■■■30-deployment-portal-r24.3-broken.yaml
10-deployment-portal-r24.1.yaml■■40-deployment-selector-change.yaml
20-deployment-portal-r24.2.yaml■■50-service-freight-portal.yaml
```

Two terminals are helpful for this lab. Terminal B is used only to watch.

3. Scenario

Marina Freight Systems (MFS) must ship release r24.2 of the *Freight Portal* - the web application customs brokers use to file declarations. The Port Authority contract sets a hard availability requirement: at no point may fewer than 4 portal replicas be serving traffic, because brokers file continuously during the 05:00-23:00 window.

The release manager has published data/release-plan.csv. It is the authority for the replica count and the rollout arithmetic; you are not free to invent values. Your job:

1. Deploy r24.1 and confirm the baseline.
2. Roll forward to r24.2 under a zero-unavailable strategy and prove the floor held.
3. When the on-call engineer pushes a hotfix tag that does not exist (r24.3), diagnose the wedge and roll back - with evidence for the incident review.

4. Step-by-step procedure

Step 1 - Read the release plan

```
column -s, -t data/release-plan.csv
```

Expected output:

revision	release	image	configmap	replicas	maxSurge	maxUnavailable	min_available_during_rollout	change_cause
1	r24.1	nginx:1.27-alpine	portal-site-r24-1	4	1	1	3	r24.1 - initial portal
2	r24.2	nginx:1.27-alpine	portal-site-r24-2	4	1	0	4	r24.2 - customs banner zero-unavailable rollout
3	r24.3	nginx:1.27-alpine-mfs-hotfix	portal-site-r24-2	4	1	0	4	r24.3 - BROKEN image tag (deliberate)

Read the two arithmetic columns before you touch the cluster:

* Revision 1 - replicas 4, maxSurge 1, maxUnavailable 1. Ceiling $4+1=5$. Floor $4-1=3$. Kubernetes may take one Pod down before a replacement is Ready.

* Revision 2 - replicas 4, maxSurge 1, maxUnavailable 0. Ceiling 5. Floor 4. Kubernetes must add a Ready Pod before it removes an old one. This satisfies the Port Authority contract.

maxSurge and maxUnavailable may not both be zero - that specification can make no progress, and the API server rejects it.

Step 2 - Namespace, site content and Service

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab07 created
```

Turn the two HTML files in data/ into ConfigMaps. The key must be index.html because that is the filename nginx serves and the readiness probe requests:

```
kubectl -n kcna-lab07 create configmap portal-site-r24-1 \
  --from-file=index.html=data/site-r24.1.html
kubectl -n kcna-lab07 create configmap portal-site-r24-2 \
  --from-file=index.html=data/site-r24.2.html
```

Expected output:

```
configmap/portal-site-r24-1 created
configmap/portal-site-r24-2 created
```

Confirm the two releases really differ:

```
diff <(kubectl -n kcna-lab07 get cm portal-site-r24-1 -o jsonpath='{.data.index\.html}') \
  <(kubectl -n kcna-lab07 get cm portal-site-r24-2 -o jsonpath='{.data.index\.html}')
```

Expected output:

```
6c6,7
< <p id="release">RELEASE=r24.1</p>
---
> <p id="release">RELEASE=r24.2</p>
> <p><strong>New:</strong> customs declaration status banner.</p>
11,12c12,13
< </ul>
< <p>build=r24.1 revision=1 strategy=RollingUpdate maxSurge=1 maxUnavailable=1</p>
---
> <li>Customs declaration status</li>
> </ul>
> <p>build=r24.2 revision=2 strategy=RollingUpdate maxSurge=1 maxUnavailable=0</p>
```

Create the Service now so it can act as a stable front door across the whole rollout:

```
kubectl apply -f manifests/50-service-freight-portal.yaml
```

Expected output:

```
service/freight-portal created
```

Step 3 - Deploy revision 1 (r24.1)

```
kubectl apply -f manifests/10-deployment-portal-r24.1.yaml
```

Expected output:

```
deployment.apps/freight-portal created
```

```
kubectl -n kcna-lab07 rollout status deployment/freight-portal --timeout=120s
```

Expected output:

```
Waiting for deployment "freight-portal" rollout to finish: 0 of 4 updated replicas are available...
Waiting for deployment "freight-portal" rollout to finish: 2 of 4 updated replicas are available...
deployment "freight-portal" successfully rolled out
```

```
kubectl -n kcna-lab07 get deploy,rs,pods
```

Expected output (hash suffixes will differ):

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/freight-portal	4/4	4	4	35s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/freight-portal-6c9f4b8d57	4	4	4	35s

NAME	READY	STATUS	RESTARTS	AGE
pod/freight-portal-6c9f4b8d57-8kq2m	1/1	Running	0	35s
pod/freight-portal-6c9f4b8d57-hd7wn	1/1	Running	0	35s
pod/freight-portal-6c9f4b8d57-p4xtl	1/1	Running	0	35s
pod/freight-portal-6c9f4b8d57-zr9vc	1/1	Running	0	35s

Note there is one ReplicaSet, not four Pods managed directly. Prove the ownership chain:

```
POD=$(kubectl -n kcna-lab07 get pods -l app=freight-portal -o name | head -1)
kubectl -n kcna-lab07 get "$POD" -o jsonpath='{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name}'; echo
kubectl -n kcna-lab07 get rs -l app=freight-portal \
  -o jsonpath='{.items[0].metadata.ownerReferences[0].kind}/{.items[0].metadata.ownerReferences[0].name}'; echo
```

Expected output:

```
ReplicaSet/freight-portal-6c9f4b8d57
Deployment/freight-portal
```

Confirm the served content is r24.1:

```
kubectl -n kcna-lab07 exec deploy/freight-portal -c web -- \
  grep RELEASE /usr/share/nginx/html/index.html
```

Expected output:

```
<p id="release">RELEASE=r24.1</p>
```

Step 4 - Watch the ceiling and floor during the r24.2 rollout

In Terminal B, start a watch that prints the Deployment status every second:

```
kubectl -n kcna-lab07 get deploy freight-portal \
  -o custom-columns=DESIRED:.spec.replicas,UPDATED:.status.updatedReplicas,TOTAL:.status.replicas,AVAILABLE:.status.availableReplicas \
  --watch
```

In Terminal A, roll forward:

```
kubectl apply -f manifests/20-deployment-portal-r24.2.yaml
```

Expected output:

```
deployment.apps/freight-portal configured
```

Terminal B prints a sequence like this (exact interleaving varies, the bounds do not):

DESIRED	UPDATED	TOTAL	AVAILABLE
4	4	4	4
4	1	5	4
4	2	5	4
4	2	5	5
4	3	5	4
4	4	5	4
4	4	4	4

Read the two invariants straight off the columns:

* TOTAL never exceeds 5 - that is the ceiling replicas + maxSurge = 4 + 1.

* AVAILABLE never drops below 4 - that is the floor replicas - maxUnavailable = 4 - 0.

Stop the watch with Ctrl-C. Back in Terminal A:

```
kubectl -n kcna-lab07 rollout status deployment/freight-portal
```

Expected output:

```
deployment "freight-portal" successfully rolled out
```

```
kubectl -n kcna-lab07 get rs -l app=freight-portal \
  -o custom-columns=NAME:.metadata.name,DESIRED:.spec.replicas,READY:.status.readyReplicas,REVISION:.metadata.annotations.deployment\kubernetes\io/revision
```

Expected output:

NAME	DESIRED	READY	REVISION
freight-portal-6c9f4b8d57	0	<none>	1
freight-portal-7d84c65f9b	4	4	2

The old ReplicaSet is retained at 0 replicas. That empty ReplicaSet *is* the rollback mechanism - there is no snapshot store, just a scaled-down controller.

Confirm the new content is being served:

```
kubectl -n kcna-lab07 exec deploy/freight-portal -c web -- \
  grep -E 'RELEASE|customs' /usr/share/nginx/html/index.html
```

Expected output:

```
<p id="release">RELEASE=r24.2</p>
<p><strong>New:</strong> customs declaration status banner.</p>
```

Step 5 - Read the rollout history

```
kubectl -n kcna-lab07 rollout history deployment/freight-portal
```

Expected output:

```
deployment.apps/freight-portal
REVISION  CHANGE-CAUSE
1          r24.1 - initial portal release
2          r24.2 - customs banner, zero-unavailable rollout
```

The CHANGE-CAUSE column is populated from the `kubernetes.io/change-cause` annotation that each manifest sets. Without it you get `<none>` and a history that tells you nothing.

Inspect one revision in detail:

```
kubectl -n kcna-lab07 rollout history deployment/freight-portal --revision=1 | head -20
```

Expected output:

```
deployment.apps/freight-portal with revision #1
Pod Template:
  Labels: ■ app=freight-portal
  ■ pod-template-hash=6c9f4b8d57
  ■ release=r24.1
  ■ tier=edge
  Annotations: ■ kubernetes.io/change-cause: r24.1 - initial portal release
  Containers:
    web:
      Image: ■ nginx:1.27-alpine
      Port: ■ 80/TCP
      Host Port: ■ 0/TCP
      Limits:
        cpu: ■ 200m
        memory: ■ 128Mi
      Requests:
        cpu: ■ 50m
        memory: ■ 64Mi
      Liveness: ■ http-get http://:http/index.html delay=10s timeout=1s period=10s #success=1
#failure=3
      Readiness: ■ http-get http://:http/index.html delay=2s timeout=1s period=3s #success=1
#failure=3
```

Note the `pod-template-hash` label. The Deployment controller adds it to the selector of every ReplicaSet it creates - that is how two ReplicaSets with otherwise identical labels stay disjoint.

```
kubectl -n kcna-lab07 get rs -l app=freight-portal \
-o jsonpath='{range .items[*]}.{metadata.name}{"  hash="}{.spec.selector.matchLabels.pod-
template-hash}{"\n"}{end}'
```

Expected output:

```
freight-portal-6c9f4b8d57 hash=6c9f4b8d57
freight-portal-7d84c65f9b hash=7d84c65f9b
```

Step 6 - Verify the current state

```
bash verification/checks.sh
```

You should see ALL CHECKS PASSED. Now break it.

5. Verification

The bundled script `verification/checks.sh` reads `data/release-plan.csv`, asserts the live Deployment matches the plan's replicas/maxSurge/maxUnavailable, confirms the rollback landed on r24.2, and confirms the Service has four EndpointSlice backends.

```
bash verification/checks.sh; echo "exit=$?"
```

The full expected transcript is in [`verification/expected-output.md`](#).

6. Failure injection

6a. A hotfix tag that does not exist wedges the rollout

The on-call engineer builds `nginx:1.27-alpine-mfs-hotfix` locally but never pushes it. Apply revision 3:

```
kubectl apply -f manifests/30-deployment-portal-r24.3-broken.yaml
```

Expected output:

```
deployment.apps/freight-portal configured
```

Watch the rollout refuse to progress:

```
kubectl -n kcna-lab07 rollout status deployment/freight-portal --timeout=150s
```

Real output - it prints the stall, then fails on the progress deadline (progressDeadlineSeconds: 120):

```
Waiting for deployment "freight-portal" rollout to finish: 1 out of 4 new replicas have been
updated...
error: deployment "freight-portal" exceeded its progress deadline
```

Look at the partial state:

```
kubectl -n kcna-lab07 get deploy,rs,pods
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/freight-portal	4/4	1	4	9m
NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/freight-portal-6c9f4b8d57	0	0	0	9m
replicaset.apps/freight-portal-7d84c65f9b	4	4	4	6m
replicaset.apps/freight-portal-5f7b9c4d68	1	1	0	2m
NAME	READY	STATUS	RESTARTS	AGE
pod/freight-portal-5f7b9c4d68-w2kfz	0/1	ImagePullBackOff	0	2m
pod/freight-portal-7d84c65f9b-4rt8x	1/1	Running	0	6m
pod/freight-portal-7d84c65f9b-9lmzq	1/1	Running	0	6m
pod/freight-portal-7d84c65f9b-mn5vd	1/1	Running	0	6m
pod/freight-portal-7d84c65f9b-tq6jw	1/1	Running	0	6m

This is the rolling update working exactly as designed. maxUnavailable: 0 forbade the controller from removing any r24.2 Pod until a new Pod became Ready. No new Pod ever became Ready. So the update stopped after the single surge Pod - and AVAILABLE is still 4. The customs brokers never noticed.

Read the real error on the wedged Pod:

```
BAD=$(kubectl -n kcna-lab07 get pods -l release=r24.3 -o name | head -1)
kubectl -n kcna-lab07 describe "$BAD" | sed -n '/^Events:$/, $p'
```

Real output:

```
Events:
  Type      Reason      Age           From          Message
  ----      -
  Normal    Scheduled   2m14s        default-scheduler Successfully assigned kcna-lab07/freight-portal-5f7b9c4d68-w2kfz to kind-control-plane
  Normal    Pulling     43s (x4 over 2m13s) kubelet        Pulling image "nginx:1.27-alpine-mfs-hotfix"
  Warning   Failed      41s (x4 over 2m11s) kubelet        Failed to pull image "nginx:1.27-alpine-mfs-hotfix": failed to pull and unpack image "docker.io/library/nginx:1.27-alpine-mfs-hotfix": failed to resolve reference "docker.io/library/nginx:1.27-alpine-mfs-hotfix": docker.io/library/nginx:1.27-alpine-mfs-hotfix: not found
  Warning   Failed      41s (x4 over 2m11s) kubelet        Error: ErrImagePull
  Normal    BackOff     5s (x7 over 2m11s) kubelet        Back-off pulling image "nginx:1.27-alpine-mfs-hotfix"
  Warning   Failed      5s (x7 over 2m11s) kubelet        Error: ImagePullBackOff
```

And the Deployment-level condition:

```
kubectl -n kcna-lab07 get deploy freight-portal \
-o jsonpath='{range .status.conditions[*]}.{.type}={.status} reason={.reason}}{\n"}{end}'
```

Expected output:

```
Available=True reason=MinimumReplicasAvailable
Progressing=False reason=ProgressDeadlineExceeded
```

Diagnosis. Available=True and Progressing=False together are the signature of a *stalled but safe* rollout: the service is up, the new version cannot start. The kubelet event names the exact cause - the tag nginx:1.27-alpine-mfs-hotfix does not resolve in docker.io/library/nginx. Nothing is wrong with the cluster, the manifest, the ConfigMap or the probes.

6b. Recover with rollout undo

```
kubectl -n kcna-lab07 rollout undo deployment/freight-portal
```

Expected output:

```
deployment.apps/freight-portal rolled back
```

```
kubectl -n kcna-lab07 rollout status deployment/freight-portal --timeout=120s
```

Expected output:

```
deployment "freight-portal" successfully rolled out
```

```
kubectl -n kcna-lab07 rollout history deployment/freight-portal
```

Expected output:

```
deployment.apps/freight-portal
REVISION  CHANGE-CAUSE
1          r24.1 - initial portal release
3          r24.3 - BROKEN image tag (deliberate)
4          r24.2 - customs banner, zero-unavailable rollout
```

Revision 2 has vanished and revision 4 has appeared. This surprises people. A rollback does not create a copy - it *re-scales the existing ReplicaSet* and re-stamps it with the next revision number. The r24.2 ReplicaSet was revision 2 and is now revision 4. Confirm:

```
kubectl -n kcna-lab07 get rs -l app=freight-portal \
-o custom-columns=NAME:.metadata.name,REVISION:.metadata.annotations.deployment\\.kubernetes\\.io/revision,DESIRED:.spec.replicas,IMAGE:.spec.template.spec.containers[0].image
```

Expected output:

NAME	REVISION	DESIRED	IMAGE
freight-portal-5f7b9c4d68	3	0	nginx:1.27-alpine-mfs-hotfix
freight-portal-6c9f4b8d57	1	0	nginx:1.27-alpine
freight-portal-7d84c65f9b	4	4	nginx:1.27-alpine

To roll back to a *specific* revision instead of the previous one:

```
kubectl -n kcna-lab07 rollout undo deployment/freight-portal --to-revision=1 --dry-run=server -o jsonpath='{.spec.template.metadata.labels.release}'; echo
```

Expected output (a server-side dry run - nothing is changed):

```
r24.1
```

Confirm the portal is serving r24.2 again:

```
kubectl -n kcna-lab07 exec deploy/freight-portal -c web -- \
grep RELEASE /usr/share/nginx/html/index.html
```

Expected output:

```
<p id="release">RELEASE=r24.2</p>
```

6c. .spec.selector is immutable

MFS wants to rename the tier label from edge to frontend. Try it on the live Deployment:

```
kubectl apply -f manifests/40-deployment-selector-change.yaml
```

Real error text:

```
The Deployment "freight-portal" is invalid: spec.selector: Invalid value: v1.LabelSelector{
  MatchLabels:map[string]string{"app":"freight-portal", "tier":"frontend"}, MatchExpressions:[
    v1.LabelSelectorRequirement(nil)}: field is immutable
```

Diagnosis. apps/v1 froze .spec.selector after creation. If it could change, the Deployment would instantly orphan every Pod it owns and start again from zero - an unannounced full outage. The supported procedure is to create a new Deployment under the new selector, shift the Service to it, then delete the old one. Label renames are not free.

7. Troubleshooting

Symptom	Cause	Fix
error: deployment "x" exceeded its progress deadline	No new Pod became Ready within progressDeadlineSeconds; usually a bad image, a failing readiness probe or an unschedulable Pod	kubectl -n <ns> describe pod -l <new release label> and read the Events block; fix the root cause or kubectl rollout undo
The Deployment "x" is invalid: spec.selector: ... field is immutable	Attempt to change .spec.selector on an existing Deployment	Create a new Deployment with the new selector, repoint the Service, then delete the old Deployment
Rollout appears to do nothing; rollout status returns immediately	kubectl apply produced no change to .spec.template; only template changes create a new revision	Confirm with kubectl diff -f <manifest>; changing .spec.replicas alone scales but never creates a revision
CHANGE-CAUSE shows <none> for every revision	The kubernetes.io/change-cause annotation was not set on the Deployment	Add the annotation in the manifest, or kubectl annotate deploy/x kubernetes.io/change-cause="..." --overwrite before the next apply
error: no rollout history found for deployment "x" after several updates	revisionHistoryLimit is too low (0 keeps nothing) and old ReplicaSets were garbage-collected	Raise .spec.revisionHistoryLimit; there is nothing to recover for revisions already collected
Pods stay ContainerCreating with configmap "portal-site-r24-2" not found	The ConfigMap was not created before the Deployment referenced it	kubectl -n kcna-lab07 create configmap portal-site-r24-2 --from-file=index.html=d ata/site-r24.2.html; the kubelet retries automatically
The Deployment is invalid: spec.strategy .rollingUpdate.maxUnavailable: Invalid value: intstr.IntOrString{...}: may not be 0 when maxSurge is 0	Both surge and unavailable set to zero - no legal move exists	Set at least one of them above zero

8. Cleanup

Delete only this lab's namespace.

```
kubectl delete namespace kcna-lab07
```

Expected output:

```
namespace "kcna-lab07" deleted
```

```
kubectl get ns kcna-lab07
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab07" not found
```

9. What you learned

- * A Deployment does not manage Pods. It manages ReplicaSets, each identified by a pod-template-hash the controller injects into the selector. Rolling forward and rolling back are both just *scaling two ReplicaSets in opposite directions*.
- * maxSurge is the ceiling above replicas; maxUnavailable is the floor below it. maxUnavailable: 0 buys a zero-downtime rollout at the cost of needing capacity for one extra Pod. They may not both be zero.
- * minReadySeconds and the readiness probe together define what "available" means - without a readiness probe, "Ready" means only "the container process started".
- * kubernetes.io/change-cause is the difference between a usable rollout history and a column of <none>.
- * rollout undo re-scales an existing old ReplicaSet and gives it a new, higher revision number; the old revision number disappears from the history.
- * revisionHistoryLimit bounds how many scaled-to-zero ReplicaSets are retained - and therefore how far back you can roll.
- * .spec.selector is immutable. Plan label schemes before the first apply.

10. Further reading

- * Deployments - <<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>>
- * Rolling update strategy and maxSurge/maxUnavailable - <<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-update-deployment>>
- * Rolling back a Deployment - <<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-back-a-deployment>>
- * kubectl rollout reference - <https://kubernetes.io/docs/reference/kubectl/generated/kubectl_rollout/>
- * Pod lifecycle and container image pull policy - <<https://kubernetes.io/docs/concepts/containers/images/>>